Ultimate Preparation Guide: Continuous Fourier Series

Part 1: Properties, Code, and Intuition

Prerequisite Utilities: Before we begin, you will need a standard way to compute the Mean Squared Error (MSE) and extract coefficients into a predictable NumPy array format for easy math. Add these to your script tomorrow:

```
import numpy as np

def calculate_mse(arr1, arr2):
    """Computes the Mean Squared Error between two complex arrays."""
    return np.mean(np.abs(arr1 - arr2)**2)

def get_coeff_array(fs_obj):
    """Extracts the dictionary of coefficients into a sorted 1D complex array."""
    # Sort keys to ensure we always go from -N to N in order
    sorted_keys = sorted(fs_obj.coeffs.keys())
    return np.array([fs_obj.coeffs[k] for k in sorted_keys], dtype=complex)
```

---

1 Layer 1: Basic Implementations (Single Properties)

1.1 Question 1: Linearity and DC Shift

Problem: Given a signal $z(t)$, create a new signal $y(t) = A \cdot z(t) + (x_0 + jy_0)$. Verify the Linearity property of the Fourier Series.

Theory & Intuition: Linearity states that if $x(t) \leftrightarrow a_k$ and $y(t) \leftrightarrow b_k$, then $Ax(t) + By(t) \leftrightarrow Aa_k + Bb_k$. Adding a constant $(x_0 + jy_0)$ is equivalent to adding a DC signal (frequency $= 0$). Therefore, *only* the c_0 coefficient should change by exactly that constant. All other coefficients are simply scaled by A.

```
# --- Setup ---
A = 2.5
dc_shift = 3.0 + 4.0j
y_signal = A * z + dc_shift

fs_y = FourierEpicycles(t, y_signal, N_HARMONICS)
fs_y.calculate_all_coefficients()

# --- Verification ---
c_y = get_coeff_array(fs_y)
c_z = get_coeff_array(fs) # Assuming fs is the original object

# Theoretical prediction
c_theoretical = A * c_z
# Find the index of the 0th harmonic (middle of the array)
zero_idx = N_HARMONICS
c_theoretical[zero_idx] += dc_shift

mse_val = calculate_mse(c_y, c_theoretical)
print(f"Q1 Linearity MSE: {mse_val}")
```

Code Explanation: We scale the original signal and add a complex constant. In the theoretical array, we scale all original coefficients by A, and manually add the ‘dc$_s hift$‘$only to the n{=}0 index$.

1.2 Question 2: Time Shifting

Problem: Shift the signal $z(t)$ in time by $t_0 = T/4$. Verify the Time Shifting property.

Theory & Intuition: $x(t - t_0) \leftrightarrow c_n e^{-jn\omega t_0}$. Intuition: If you delay a signal, you aren't changing its shape, so the *magnitudes* of the frequencies ($|c_n|$) stay exactly the same. However, to reconstruct the delayed shape, every rotating epicycle must start at a different angle. Higher frequencies spin faster, so they need a larger phase shift to account for the same time delay t_0. Hence, the phase shift $-n\omega t_0$ depends on n.

```
# --- Setup ---
t0 = fs.T / 4.0

# Method 1: Using np.interp (Safest for continuous periodic signals)
# We evaluate z at (t - t0). We use modulo fs.T to handle wraparound.
t_shifted = (t - t0) % fs.T
# np.interp requires strictly increasing x-values, so we must separate real/imag
y_real = np.interp(t_shifted, t, z.real)
y_imag = np.interp(t_shifted, t, z.imag)
y_signal = y_real + 1j * y_imag

# Alternative Method 2: Using np.roll (Only works if t0 is an exact multiple of dt)
# shift_idx = int(t0 / (t[1] - t[0]))
# y_signal = np.roll(z[:-1], shift_idx)
# y_signal = np.append(y_signal, y_signal[0]) # restore closed interval

fs_y = FourierEpicycles(t, y_signal, N_HARMONICS)
fs_y.calculate_all_coefficients()

# --- Verification ---
c_y = get_coeff_array(fs_y)
c_z = get_coeff_array(fs)

n_arr = np.arange(-N_HARMONICS, N_HARMONICS + 1)
c_theoretical = c_z * np.exp(-1j * n_arr * fs.omega * t0)

print(f"Q2 Time Shift MSE: {calculate_mse(c_y, c_theoretical)}")
```

Code Explanation: ‘np.interp‘ is the most robust way to shift a signal in time because it doesn't rely on t_0 perfectly aligning with your array indices. We use modulo ‘

1.3 Question 3: Time Reversal and Conjugation

Problem: Create a time-reversed signal $y(t) = z(-t)$ and a conjugated signal $w(t) = z^*(t)$. Verify their properties.

Theory & Intuition: Time Reversal: $x(-t) \leftrightarrow c_{-n}$. Playing a signal backwards reverses the direction of all the rotating epicycles. Conjugation: $x^*(t) \leftrightarrow c^*_{-n}$. Conjugating the spatial coordinates flips the shape over the x-axis.

```
# --- Setup ---
# Time Reversal: Reverse the array.
# Because t[0]=0 and t[-1]=T are the same point, we reverse the inner elements.
y_signal = np.zeros_like(z)
y_signal[0] = z[0]
y_signal[1:] = z[:0:-1] # slice from end down to index 1

# Conjugation
w_signal = np.conj(z)
```

```
fs_y = FourierEpicycles(t, y_signal, N_HARMONICS)
fs_y.calculate_all_coefficients()
fs_w = FourierEpicycles(t, w_signal, N_HARMONICS)
fs_w.calculate_all_coefficients()

# --- Verification ---
c_y = get_coeff_array(fs_y)
c_w = get_coeff_array(fs_w)
c_z = get_coeff_array(fs)

# Theoretical: c_{-n} is just the reversed coefficient array!
c_theoretical_y = c_z[::-1]
c_theoretical_w = np.conj(c_z[::-1])

print(f"Q3 Time Reversal MSE: {calculate_mse(c_y, c_theoretical_y)}")
print(f"Q3 Conjugation MSE: {calculate_mse(c_w, c_theoretical_w)}")
```

1.4 Question 4: Frequency Shifting (Modulation)

Problem: Multiply the signal by a complex exponential $y(t) = e^{jM\omega t}z(t)$ where $M = 5$. Verify the property.

Theory & Intuition: $e^{jM\omega t}x(t) \leftrightarrow c_{n-M}$. Intuition: Multiplying by a complex exponential in time "spins" the entire drawing. In the frequency domain, this shifts all the coefficients to the right by M slots.

```
# --- Setup ---
M = 5
y_signal = z * np.exp(1j * M * fs.omega * t)

fs_y = FourierEpicycles(t, y_signal, N_HARMONICS)
fs_y.calculate_all_coefficients()

# --- Verification ---
c_y = get_coeff_array(fs_y)
c_z = get_coeff_array(fs)

# Theoretical: Shift the array to the right by M.
# Note: Because we have a finite N, shifting drops the last M elements
# and we don't know the new first M elements. We verify only the overlapping part.
c_y_valid = c_y[M:]           # The computed shifted coefficients
c_theo_valid = c_z[:-M]       # The original coefficients

print(f"Q4 Frequency Shift MSE: {calculate_mse(c_y_valid, c_theo_valid)}")
```

---

2 Layer 2: Harder (Calculus, Scaling, & Combinations)

2.1 Question 5: Time Scaling

Problem: Compress the signal in time by a factor of $\alpha = 2$, so $y(t) = z(2t)$. Verify the Time Scaling property.

Theory & Intuition: $x(\alpha t) \leftrightarrow c_n$. Intuition: Playing a video at 2x speed doesn't change the frames of the video (the shape, c_n), it only changes how fast they play (the fundamental frequency becomes $\alpha\omega$, and the period becomes T/α).

```
# --- Setup ---
alpha = 2.0
# The new period is T / alpha
T_new = fs.T / alpha
```

```
# We must create a new time array for the new period
t_new = np.linspace(0, T_new, len(t))

# The signal values themselves don't change, they just happen over a shorter time!
y_signal = z.copy()

# Initialize with the NEW time array
fs_y = FourierEpicycles(t_new, y_signal, N_HARMONICS)
fs_y.calculate_all_coefficients()

# --- Verification ---
c_y = get_coeff_array(fs_y)
c_z = get_coeff_array(fs)

# Theoretical: The coefficients are EXACTLY the same.
print(f"Q5 Time Scaling MSE: {calculate_mse(c_y, c_z)}")
# Also verify the frequency changed
print(f"Original Omega: {fs.omega}, New Omega: {fs_y.omega} (Should be {alpha * fs.
    omega})")
```

Code Explanation: This is a trick question regarding OOP. You do not interpolate the signal array. You simply pass the exact same 'z' array into 'FourierEpicycles', but you pass a *compressed time array* 't_new'.*Theclasswillautomaticallycalculatethenew*T*and*ω.

2.2 Question 6: Differentiation Property

Problem: Compute the derivative of the signal $y(t) = \frac{d}{dt}z(t)$. Verify the differentiation property.

Theory & Intuition: $\frac{dx(t)}{dt} \leftrightarrow jn\omega c_n$. Intuition: The derivative measures the rate of change. High-frequency epicycles spin faster, so their rate of change is much larger (hence multiplying by $n\omega$). The j represents a 90° phase shift (since the derivative of sine is cosine).

```
# --- Setup ---
# Numerical differentiation using np.gradient
# Since t is uniformly spaced, we can just pass the spacing dt
dt = t[1] - t[0]
y_signal = np.gradient(z, dt)

fs_y = FourierEpicycles(t, y_signal, N_HARMONICS)
fs_y.calculate_all_coefficients()

# --- Verification ---
c_y = get_coeff_array(fs_y)
c_z = get_coeff_array(fs)

n_arr = np.arange(-N_HARMONICS, N_HARMONICS + 1)
c_theoretical = 1j * n_arr * fs.omega * c_z

print(f"Q6 Differentiation MSE: {calculate_mse(c_y, c_theoretical)}")
```

Alternative Method: Instead of 'np.gradient', you could use finite differences: '$y_signal = np.diff(z)/dt$', *but*'$np.diff$'*reducesthearraysizeby*1.*Youwouldhavetoappendavaluetokeeptheshapeconsistent*

2.3 Question 7: Parseval's Relation (Energy Conservation)

Problem: Verify Parseval's theorem for the signal $z(t)$.

Theory & Intuition: $\frac{1}{T}\int_0^T |x(t)|^2 dt = \sum_{n=-\infty}^{\infty} |c_n|^2$. Intuition: The total energy of the signal in the time domain must exactly equal the sum of the energies of all its individual frequency components. (Note: Because we only compute up to N harmonics, the frequency side will be slightly smaller than the time side. The MSE won't be exactly 0, but it should be very close if N is large).

```
# Time domain power (using np.trapezoid as required)
power_time = np.trapezoid(np.abs(z)**2, t) / fs.T

# Frequency domain power
c_z = get_coeff_array(fs)
power_freq = np.sum(np.abs(c_z)**2)

print(f"Q7 Parseval's - Time Power: {power_time:.4f}, Freq Power: {power_freq:.4f}"
    )
print(f"Difference (Energy in truncated harmonics): {power_time - power_freq:.6f}")
```

2.4 Question 8: Conjugate Symmetry and Even/Odd Decomposition

Problem: Extract the real part of the signal $x(t) = \Re\{z(t)\}$. Verify that its coefficients exhibit conjugate symmetry ($c_n = c_{-n}^*$). Then, decompose $x(t)$ into Even and Odd parts and verify their coefficient properties.

Theory & Intuition: If a signal is purely real, its negative frequencies must perfectly cancel out the imaginary parts of its positive frequencies. Therefore, c_{-n} must be the complex conjugate of c_n. Furthermore, the Even part of a real signal corresponds to the Real part of c_n, and the Odd part corresponds to the Imaginary part of c_n.

```
# --- Setup ---
x_real = z.real
fs_real = FourierEpicycles(t, x_real, N_HARMONICS)
fs_real.calculate_all_coefficients()
c_real = get_coeff_array(fs_real)

# 1. Verify Conjugate Symmetry: c_n == conj(c_{-n})
# c_real[::-1] gives us the array reversed (which maps n to -n)
mse_sym = calculate_mse(c_real, np.conj(c_real[::-1]))
print(f"Q8 Conjugate Symmetry MSE: {mse_sym}")

# 2. Even/Odd Decomposition
# x_even(t) = (x(t) + x(-t)) / 2
x_rev = np.zeros_like(x_real)
x_rev[0] = x_real[0]
x_rev[1:] = x_real[:0:-1]

x_even = (x_real + x_rev) / 2.0
x_odd = (x_real - x_rev) / 2.0

fs_even = FourierEpicycles(t, x_even, N_HARMONICS)
fs_even.calculate_all_coefficients()
fs_odd = FourierEpicycles(t, x_odd, N_HARMONICS)
fs_odd.calculate_all_coefficients()

c_even = get_coeff_array(fs_even)
c_odd = get_coeff_array(fs_odd)

# Theoretical: c_even should be purely real, c_odd purely imaginary
# We compare them to the real and imaginary parts of the original c_real
mse_even = calculate_mse(c_even, c_real.real)
mse_odd = calculate_mse(c_odd, 1j * c_real.imag)

print(f"Q8 Even Part MSE: {mse_even}")
print(f"Q8 Odd Part MSE: {mse_odd}")
```

---

3 Layer 3: Very Hard (Convolutions, Integrations, Multi-layered)

3.1 Question 9: Periodic Convolution

Problem: Given two signals $x(t)$ and $y(t)$ (let's use $z(t)$ and a shifted version of $z(t)$), compute their periodic convolution $w(t) = \int_0^T x(\tau)y(t-\tau)d\tau$. Verify the convolution property.

Theory & Intuition: $x(t) \circledast y(t) \leftrightarrow T \cdot c_n^{(x)} c_n^{(y)}$. Convolution in the time domain is equivalent to multiplication in the frequency domain (scaled by T). Because you cannot use 'np.fft' or 'scipy.signal.convolve' (which uses FFT under the hood), you must implement the convolution integral manually using 'np.trapezoid'.

```
# --- Setup ---
x_sig = z
# Let y_sig be a simple sine wave to make convolution fast and clean
y_sig = np.sin(fs.omega * t)

# Manual O(N^2) Periodic Convolution using np.trapezoid
w_sig = np.zeros_like(t, dtype=complex)
# We need y(-tau).
y_rev = np.zeros_like(y_sig)
y_rev[0] = y_sig[0]
y_rev[1:] = y_sig[:0:-1]

for i in range(len(t)):
    # Shift y(-tau) to the right by i steps to get y(t - tau)
    y_shifted = np.roll(y_rev, i)
    # Integrate x(tau) * y(t - tau) d_tau
    w_sig[i] = np.trapezoid(x_sig * y_shifted, t)

fs_y = FourierEpicycles(t, y_sig, N_HARMONICS)
fs_y.calculate_all_coefficients()
fs_w = FourierEpicycles(t, w_sig, N_HARMONICS)
fs_w.calculate_all_coefficients()

# --- Verification ---
c_x = get_coeff_array(fs)
c_y = get_coeff_array(fs_y)
c_w = get_coeff_array(fs_w)

c_theoretical = fs.T * c_x * c_y

print(f"Q9 Periodic Convolution MSE: {calculate_mse(c_w, c_theoretical)}")
```

Code Explanation: We manually implement the convolution integral. 'np.roll(y_rev, i)'$perfectlysimulatesy(t - \tau)becausethesignalisperiodicanduniformlysampled.Weintegratetheproductusing$'$np.trapezoid$'$.

3.2 Question 10: Multiplication in Time

Problem: Multiply two signals in the time domain $w(t) = x(t)y(t)$. Verify the property.

Theory & Intuition: $x(t)y(t) \leftrightarrow \sum_{l=-\infty}^{\infty} c_l^{(x)} c_{n-l}^{(y)}$. Multiplication in time is convolution in frequency! The coefficients of the multiplied signal are the discrete convolution of the individual coefficient arrays.

```
# --- Setup ---
x_sig = z
y_sig = np.cos(2 * fs.omega * t) # A 2nd harmonic cosine
w_sig = x_sig * y_sig

fs_y = FourierEpicycles(t, y_sig, N_HARMONICS)
fs_y.calculate_all_coefficients()
fs_w = FourierEpicycles(t, w_sig, N_HARMONICS)
```

```
fs_w.calculate_all_coefficients()

# --- Verification ---
c_x = get_coeff_array(fs)
c_y = get_coeff_array(fs_y)
c_w = get_coeff_array(fs_w)

# Discrete convolution of coefficients.
# np.convolve does exactly sum(a[l] * b[n-l]).
# Mode 'same' keeps the array size 2N+1.
c_theoretical = np.convolve(c_x, c_y, mode='same')

print(f"Q10 Multiplication MSE: {calculate_mse(c_w, c_theoretical)}")
```

3.3 Question 11: The Integration Property

Problem: Compute the integral of the signal $y(t) = \int_{-\infty}^{t} x(\tau)d\tau$. Verify the property.

Theory & Intuition: $\int x(t)dt \leftrightarrow \frac{c_n}{jn\omega}$. Integration is the opposite of differentiation, so we divide by $jn\omega$. **CRITICAL CATCH:** This property is *only* valid if $c_0 = 0$ (the signal has zero mean). If $c_0 \neq 0$, integrating a constant yields a ramp function ($c_0 t$), which goes to infinity and is no longer periodic!

```
import scipy.integrate

# --- Setup ---
# 1. We MUST remove the DC component first!
c0 = fs.coeffs[0]
x_zero_mean = z - c0

# 2. Integrate using cumulative trapezoid
# initial=0 ensures the output array is the same size as t
y_signal = scipy.integrate.cumulative_trapezoid(x_zero_mean, t, initial=0)

fs_y = FourierEpicycles(t, y_signal, N_HARMONICS)
fs_y.calculate_all_coefficients()

# --- Verification ---
c_y = get_coeff_array(fs_y)
c_x = get_coeff_array(fs) # Original coefficients

n_arr = np.arange(-N_HARMONICS, N_HARMONICS + 1)
c_theoretical = np.zeros_like(c_x, dtype=complex)

# Apply formula ONLY for n != 0 to avoid division by zero
for i, n in enumerate(n_arr):
    if n != 0:
        c_theoretical[i] = c_x[i] / (1j * n * fs.omega)
    else:
        # The DC component of the integrated signal depends on the constant of
    integration.
        # We just copy the computed one to ignore it in the MSE calculation.
        c_theoretical[i] = c_y[i]

print(f"Q11 Integration MSE: {calculate_mse(c_y, c_theoretical)}")
```

Code Explanation: We use 'scipy.integrate.cumulative$_t rapezoid$'$toperformtherunningintegral.Wemustma$

3.4 Question 12: The Ultimate Combo

Problem: Given $z(t)$, construct $y(t) = \frac{d}{dt}[z(t - t_0)] \cdot e^{j\omega_0 t}$. Find the theoretical coefficients and verify.

Theory & Intuition: Apply properties sequentially: 1. Shift: $c_n^{(1)} = c_n e^{-jn\omega t_0}$ 2. Differentiate: $c_n^{(2)} = jn\omega \cdot c_n^{(1)} = jn\omega c_n e^{-jn\omega t_0}$ 3. Frequency Shift (by $M = \omega_0/\omega$): $c_n^{(3)} = c_{n-M}^{(2)} = j(n-M)\omega c_{n-M} e^{-j(n-M)\omega t_0}$.

```
# --- Setup ---
t0 = fs.T / 8.0
M = 2

# 1. Shift
t_shifted = (t - t0) % fs.T
z_shift = np.interp(t_shifted, t, z.real) + 1j * np.interp(t_shifted, t, z.imag)
# 2. Differentiate
dt = t[1] - t[0]
z_diff = np.gradient(z_shift, dt)
# 3. Frequency Shift
y_signal = z_diff * np.exp(1j * M * fs.omega * t)

fs_y = FourierEpicycles(t, y_signal, N_HARMONICS)
fs_y.calculate_all_coefficients()

# --- Verification ---
c_y = get_coeff_array(fs_y)
c_z = get_coeff_array(fs)
n_arr = np.arange(-N_HARMONICS, N_HARMONICS + 1)

# Theoretical calculation
c_theo_full = 1j * n_arr * fs.omega * c_z * np.exp(-1j * n_arr * fs.omega * t0)
# Shift the theoretical array by M
c_theo_valid = c_theo_full[:-M]
c_y_valid = c_y[M:]

print(f"Q12 Ultimate Combo MSE: {calculate_mse(c_y_valid, c_theo_valid)}")
```